

Documentazione del Sistema

Casa della Salute

Progetto di Ingegneria del Software

Corso di Laurea in Bioinformatica

Università degli Studi di Verona

Barichello Rebecca (VR471455)

Kaur Taniya (VR481429)

Magrograssi Federico (VR472578)

Contents

1 Introduzione

Il sistema software in questione è stato creato per la gestione dei servizi erogati da una Casa della Salute, come visite mediche, prelievi e medicazioni.

Si suppone che il software venga distribuito ai pazienti e al personale della Casa interessati, e che sia usufruibile tramite appositi terminali presenti nella struttura.

Gli utenti, per poter usufruire dei servizi, dovranno essere in possesso delle loro credenziali. Queste verranno generate dal personale di segreteria e comunicate agli utenti stessi.

Obiettivi del sistema

Il sistema è progettato per gestire le attività di una Casa della Salute, dove operano più medici di base, supportati da infermieri per medicazioni e prelievi, e un ufficio di segreteria comune. La struttura ospita:

- 3 ambulatori per visite e prelievi ad adulti,
- 1 ambulatorio pediatrico,

Funzionalità principali

Il sistema permette di:

- Registrare e gestire i dati di medici, infermieri e pazienti.
- Gestire le prenotazioni per visite, inviando mail di conferma.
- Mantenere una storia delle visite e delle prestazioni effettuate.
- Fornire resoconti annuali delle prestazioni erogate ai pazienti.

Autenticazione e accesso

Il sistema permette l'autenticazione ai pazienti tramite le credenziali fornite dalla segreteria. Il personale medico e infermieristico, invece, può accedere tramite le proprie credenziali personali, sempre nella sezione login.

2 Casi d'uso

In questa sezione verranno inseriti i diagrammi dei casi d'uso per ciascuna categoria di utente. Per ogni figura utente (medico, segreteria, paziente), si illustrano le principali interazioni con il sistema.

2.1 Personale medico

Il personale medico può autenticarsi nel sistema, visualizzare le visite programmate, accedere ai dati dei propri pazienti, inserire l'esito delle visite e inviare notifiche al paziente. [Diagramma dei casi d'uso da inserire qui]

2.2 Segreteria

La segreteria gestisce la registrazione di nuovi pazienti e operatori sanitari, la modifica o eliminazione dei dati anagrafici, e la gestione delle prenotazioni delle visite. Diagramma dei casi d'uso da inserire qui

2.3 Paziente

Il paziente può autenticarsi, prenotare visite, visualizzare gli esiti delle stesse, prenotare prelievi e medicazioni, e consultare lo storico delle prestazioni ricevute. [Diagramma dei casi d'uso da inserire qui]

3 Diagrammi delle attività

Come per i diagrammi visti in precedenza, si assume che le operazioni possano essere annullate in qualsiasi momento, per esempio eseguendo il logout dal sistema.

Per questioni di leggibilità, si omette la possibilità di visualizzazione e modifica del profilo da parte degli utenti.

3.1 Personale medico

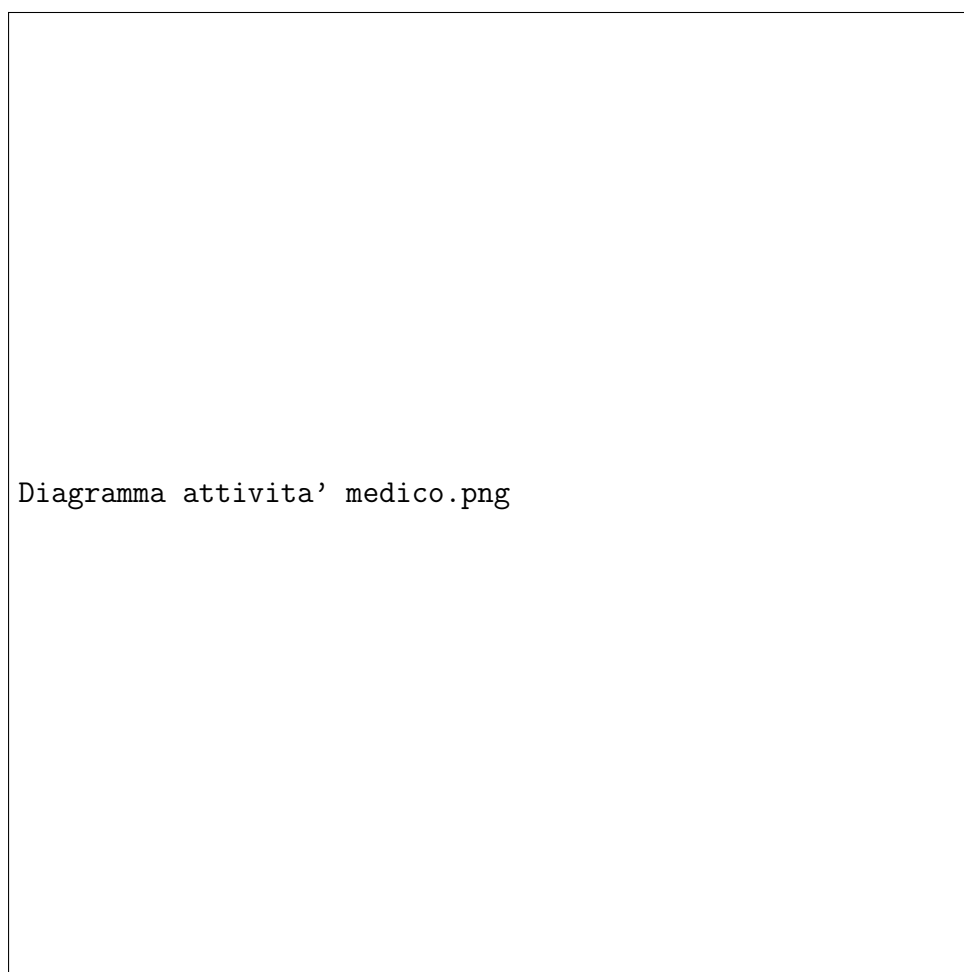


Figure 1: Diagramma di attività per il personale medico

3.2 Segreteria

Il diagramma delle attività della segreteria descrive il processo completo che va dall'autenticazione dell'utente alla gestione delle anagrafiche di pazienti e personale sanitario, alla pianificazione e modifica delle prenotazioni. [Diagramma da inserire qui]

3.3 Paziente

Il diagramma delle attività del paziente descrive il flusso che inizia con l'autenticazione, prosegue con la prenotazione di una visita o di una prestazione, la consultazione degli

esiti e la visualizzazione dello storico.
Diagramma da inserire qui

4 Sviluppo: progetto dell'architettura ed implementazione del sistema

Note sul processo di sviluppo

Il processo di sviluppo ha seguito un approccio agile e incrementale. Si assume che solo i pazienti adulti possano essere registrati dal personale di segreteria, mentre i pazienti pediatrici sono inseriti da pazienti maggiorenni.

Gli adulti che non sono maggiorenni non possono avere pazienti pediatrici a loro carico. Quando un paziente pediatrico compie 14 anni, e diventa un ex paziente pediatrico.

Il pediatra può visualizzare gli appuntamenti degli ex pazienti pediatrici e aggiornare l'esito delle visite.

Note sul paziente maggiorenne

Un paziente maggiorenne deve obbligatoriamente essere associato a un medico per poter essere registrato a sistema. Questa scelta è legata alla natura prototipale del progetto. Il paziente maggiorenne potrà poi inserire un paziente pediatrico a suo carico

5 Modelli architetturali e progettazione ad alto livello

Il nostro progetto è stato sviluppato utilizzando il framework Django, che si basa su un'architettura denominata **Model-View-Template (MVT)**. Questo modello è una variante del più noto pattern MVC, adattata alle caratteristiche del framework stesso. Grazie a questa struttura, è possibile separare in modo netto la logica di gestione dei dati, l'elaborazione delle richieste e la visualizzazione lato utente.

4.2.1 Il pattern Model-View-Template (MVT)

Nel pattern MVT, ogni componente ha un ruolo specifico:

- **Model:** definisce la struttura dei dati e la logica di accesso al database. In Django, i model sono scritti in Python all'interno dei file `models.py`, e corrispondono direttamente alle tabelle nel database.
- **View:** contiene la logica dell'applicazione. Si occupa di gestire le richieste HTTP, interagire con i model, elaborare i dati e restituire una risposta, generalmente tramite un template HTML. Le view sono definite nel file `views.py`.
- **Template:** rappresenta la parte visibile all'utente. I template sono file HTML dinamici che permettono di visualizzare i dati elaborati dalle view, usando il linguaggio di templating integrato in Django.

In sintesi, i dati vengono gestiti dai model, elaborati dalle view e presentati all'utente tramite i template. Questo approccio ci ha permesso di organizzare il codice in modo modulare e facilmente manutenibile.

Nel nostro progetto, abbiamo creato diversi modelli per rappresentare utenti, medici, pazienti, prenotazioni e prestazioni. Ogni funzionalità è stata sviluppata come view associata a uno specifico URL, e i dati sono stati presentati attraverso template responsivi.

4.3 Design pattern utilizzati

Durante lo sviluppo abbiamo fatto uso di alcuni design pattern che Django supporta nativamente o che si adattano bene alla sua struttura:

- **Singleton**: questo pattern viene utilizzato in contesti in cui serve garantire un'unica istanza condivisa, ad esempio nella gestione centralizzata delle impostazioni di configurazione.
- **Observer**: Django include un sistema di segnali, come `post_save` e `pre_delete`, che ci ha permesso di eseguire azioni automatiche al verificarsi di determinati eventi (come l'invio di notifiche dopo la prenotazione o la modifica di un esito).
- **Adapter**: in alcuni casi abbiamo integrato librerie esterne adattandole alle esigenze del progetto, in particolare per la validazione dei form e per la gestione dell'interfaccia utente.

4.4 Librerie di terze parti utilizzate

Oltre ai moduli standard di Django e Python, abbiamo utilizzato alcune librerie esterne per migliorare l'efficienza nello sviluppo e la qualità del software:

- **Django**: il framework principale, utilizzato per costruire l'intera architettura web.
- **python-dotenv**: per la gestione sicura delle variabili d'ambiente.
- **crispy-forms**: per migliorare la resa grafica e la personalizzazione dei form HTML.
- **pytest-django**: per scrivere e gestire i test automatici in modo semplice ed efficace.
- **coverage.py**: per valutare la copertura del codice da parte dei test e individuare le parti non ancora verificate.

Versioning del progetto

Durante lo sviluppo del progetto si è utilizzato `git` come sistema di versionamento. Il repository è stato ospitato su GitHub, permettendo una gestione centralizzata del codice sorgente e facilitando la collaborazione tra i membri del team. Ogni modifica al codice è stata registrata tramite commit, e sono stati creati branch per le funzionalità in fase di sviluppo. Questo approccio ha garantito una storia chiara delle modifiche e ha facilitato il processo di revisione del codice.

<https://github.com/Taniyakaur/Project>

6 Installazione e configurazione del sistema

7 Installazione e configurazione del sistema

Questa sezione descrive i passaggi necessari per installare e configurare il progetto localmente su una macchina con sistema operativo Windows, macOS o Linux.

Requisiti

Per poter eseguire correttamente il progetto sono richiesti i seguenti strumenti:

- Python 3.10 o superiore
- pip (Python package installer)
- Git
- Un ambiente virtuale (consigliato: `venv`)

Clonazione del repository

Aprire un terminale e clonare il progetto da GitHub:

```
git clone https://github.com/Taniyakaur/Project.git cd Project
```

Creazione dell'ambiente virtuale

Per creare e attivare un ambiente virtuale:

- Su Windows: `python -m venv venv venv`
- Su macOS/Linux: `python3 -m venv venv source venv/bin/activate`

Installazione delle dipendenze

Con l'ambiente attivo, installare i pacchetti necessari:

```
pip install -r requirements.txt
```

Configurazione delle variabili d'ambiente

Creare un file `.env` nella root del progetto con le seguenti variabili (personalizzare i valori):

```
DEBUG=True SECRET_KEY = chiave_sigaretissima ALLOWED_HOSTS = localhost, 127.0.0.1
```

Migrazione del database

Applicare le migrazioni iniziali:

```
python manage.py makemigrations python manage.py migrate
```

Creazione di un superutente (opzionale)

Per accedere al pannello di amministrazione:

```
python manage.py createsuperuser
```

Avvio del server di sviluppo

Eseguire il server:

```
python manage.py runserver
```

Il sistema sarà accessibile all'indirizzo `http://127.0.0.1:8000`

Accesso al sistema

Per accedere al sistema, utilizzare le credenziali fornite dalla segreteria per i pazienti, o quelle create per il personale medico e infermieristico.

8 Test e validazione

5.1 Unit test

Abbiamo eseguito test di tipo unitario su diverse componenti del sistema, in particolare sui modelli e sulle view. Per questo abbiamo utilizzato il framework di test integrato in Django, basato su `unittest`, e in alcuni casi `pytest-django` per una maggiore flessibilità.

Gli unit test sono stati scritti all'interno dei file `tests.py` presenti in ciascuna app Django.

Esempi di test effettuati:

- Validazione dei form (es. campi obbligatori, formati email, password, ecc.).
- Verifica della corretta creazione e salvataggio dei modelli nel database.
- Controllo che determinate view richiedano l'autenticazione.
- Verifica della logica di assegnazione del medico e delle prenotazioni.

5.2 Test manuali degli sviluppatori

Abbiamo svolto test manuali sistematici simulando le azioni tipiche degli utenti:

- Login e logout di utenti di diversi ruoli (medico, segreteria, paziente).
- Registrazione nuovi utenti e modifica dei dati anagrafici.
- Prenotazione e cancellazione di prestazioni.
- Inserimento di esiti e verifica della notifica al paziente.
- Gestione di situazioni particolari (es. doppia prenotazione nello stesso orario).

5.3 Copertura del codice e strumenti

Per misurare la copertura dei test automatizzati abbiamo utilizzato `coverage.py`, uno strumento che mostra quali righe di codice sono state eseguite durante i test. Questo ci ha permesso di identificare porzioni di codice non ancora testate.

5.4 Proposte per test futuri

In futuro, sarebbe utile integrare:

- Test end-to-end con **Selenium** o **Playwright**, per simulare l'interazione dell'utente con l'interfaccia web.
- Test di carico e prestazioni con **Locust** o **JMeter**, per verificare il comportamento del sistema sotto stress.